Eliminating Institutional Amnesia in SRE: How We Built Echo-Core with Persistent Biomimetic Memory and Groq

Author: TeamOps
Shrestha Srivastava
Sudipti Tewari
Anushka Srivastava
Disharee Das
Sneha Paul
Shachi Rai
Project: Echo-Core Incident Response Agent
Hackathon: HackwithBengaluru 2.0
Category: Autonomous DevOps & Persistent AI Agents

The 3:00 AM On-Call Dilemma: Institutional Amnesia

Every software engineer who has ever carried a production pager knows this nightmare: It is 3:14 AM on a Saturday. PagerDuty starts blaring. The payments service is throwing 500s at a 12% error rate, checkout latency has ballooned to 4.2 seconds, and executive Slack channels are blowing up.

You stumble to your laptop, bleary-eyed. Your immediate instinct—the same instinct nearly every engineer has—is: *"Let's restart the pods."*

You run kubectl rollout restart deploy/payments.

Within 45 seconds, the entire infrastructure collapses. What was a localized connection pool saturation on payments instantly triggers a violent, thundering connection stampede against PostgreSQL. All available connection slots are exhausted. The database locks up. Now, orders, auth, and search are dead too. A localized SEV-2 just cascaded into an existential company-wide SEV-1.

Here is the tragedy: **Your team already learned this exact lesson three months ago.**

Senior SRE Sarah spent six hours debugging this in November. She wrote a Slack post-mortem noting that restarting payments under load crashes HikariCP connection limits and that the only safe fix is scaling replicas with a pre-warmed pool limit patch. But that knowledge is buried inside a Slack thread nobody remembers, a closed Jira ticket nobody queried, or inside the head of an engineer who left the company last month.

This systemic flaw is what we call **Institutional Amnesia**:

1. Teams repeatedly attempt the **same failed fixes**, turning minor alerts into cascading outages.
2. Root causes recur across deployments because historical context is lost.
3. Every on-call engineer operates in a vacuum, forced into trial-and-error under high adrenaline.

To solve this, our team built **Echo-Core**: an autonomous incident response agent with persistent, biomimetic memory powered by **Vectorize Hindsight** and an enterprise-grade **Groq LLM** resilience engine.

Why Generic AI Agents Fail at Incident Response

When teams try to plug generic LLMs (zero-memory ChatGPT, raw Claude, or basic RAG) into SRE runbooks, the results in production can be dangerous.

Generic LLMs suffer from two fatal weaknesses during live incidents:

1. **Recency and Specificity Blindness:** A standard LLM has general knowledge of Kubernetes and databases, but zero awareness of *your* cluster's quirks. It will cheerfully recommend kubectl rollout restart because that is textbook advice, completely unaware that in *your* system, that command triggers a catastrophic database lock.

2. **Failure Ignorance:** Standard vector embeddings retrieve text similar to the problem. But they do not distinguish between *mitigations that succeeded* and *mitigations that caused disasters*. An LLM without structured temporal memory will happily repeat a previously disastrous action.

Architecture: How Echo-Core Works

Echo-Core bridges live production telemetry, an autonomous agent loop, multi-tier LLM inference, and persistent biomimetic memory.

The Three Pillars of Persistent Memory with Vectorize Hindsight

Instead of naive vector chunking, Echo-Core implements the three foundational operations of Vectorize Hindsight:

1. recall(): Multi-Strategy Hybrid Retrieval

When an alert triggers, Echo-Core initiates a parallel hybrid search across the team's persistent memory bank (acme-sre):

- **Semantic Vector Similarity:** Discovers past incidents with similar symptom vectors and log traces.
- **BM25 Lexical Keyword Search:** Pinpoints exact error codes (e.g., HikariPool-1 - Connection is not available, OOMKilled exit 137).
- **Graph Proximity:** Understands the dependency graph between microservices (e.g., how payments saturates postgres).

2. reflect(): Cross-Incident Synthesis & Anti-Pattern Discovery

Rather than merely pasting old tickets, Echo-Core's agent uses reflection to synthesize patterns across incidents:

- *"Every time latency spikes on payments after a deployment while error rates remain under 2%, pod restarts cause pool exhaustion."*

- It explicitly extracts **Failed Fix Warnings** and flags them prominently before an engineer can execute them.

3. retain(): Continuous Knowledge Ingestion

When an engineer resolves an incident in the Command Center, Echo-Core prompts for resolution notes and whether the agent's suggestion worked. The outcome is ingested into Hindsight in the background. The next time a similar alert occurs, the agent is already smarter.

Safety First: The 41-Pattern Pre-Flight Guardrail Engine

In live incident management, **safety must precede automation**. Even the best model should never be given unchecked execution rights over production clusters.

We implemented a dynamic pre-flight safety engine in backend/agent.py covering **41 documented historical disaster patterns** across our microservices:

Service	Dangerous Anti-Pattern	Safe Proven Alternative
payments	kubectl rollout restart (triggers connection stampede)	kubectl scale deploy/payments --replicas=6
postgres	VACUUM FULL (exclusive table locking during peak traffic)	Standard incremental VACUUM with lock timeout
redis	FLUSHDB / FLUSHALL (unbounded cache stampede)	Granular prefix eviction with TTL jitter
queue	Blind DLQ redrive without schema alignment	Poison message isolation + replay throttled at 50 rps
ml-ranker	StartupProbe timeout inflation (masks CUDA memory leaks)	Worker pool recycling with heap snapshot capture

If an engineer attempts to run a dangerous command, Echo-Core displays a bright crimson warning modal, explains the historical precedent (citing past incident IDs and resolvers), and requires typing the exact service name to confirm or switch to the proven alternative.

Tri-Model Resilience: Zero Outage Inference via Groq

Incident response systems cannot go down during an incident. If your primary LLM rate-limits or experiences an outage, your on-call agent becomes useless.

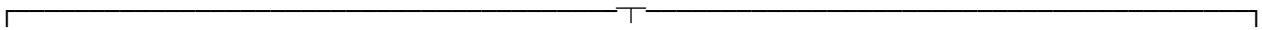
To eliminate single points of failure, Echo-Core implements a **Tri-Model Fallback Chain** in backend/llm.py powered by Groq's low-latency LPU hardware:

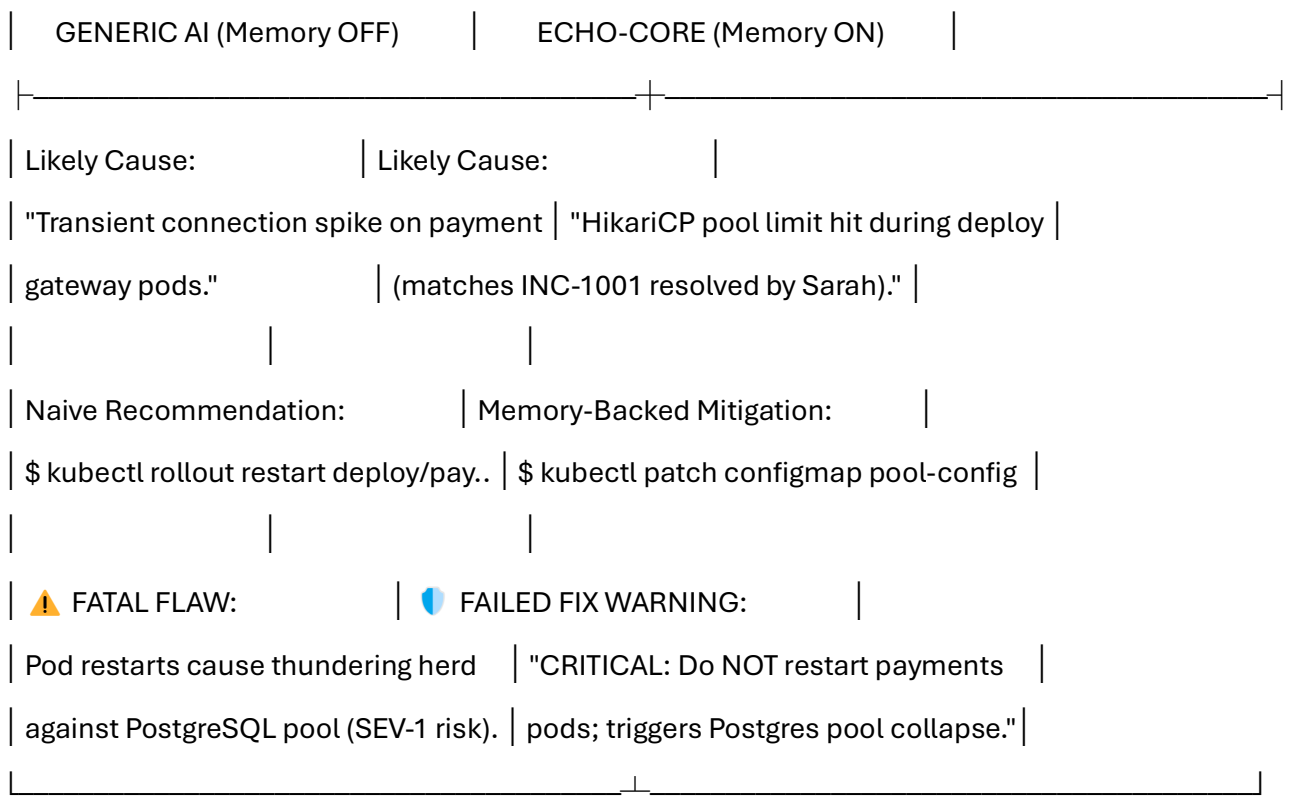
1. **Primary (openai/gpt-oss-120b):** 120-billion parameter reasoning model for deep contextual analysis and root-cause synthesis.
2. **Secondary Fallback (qwen/qwen3.8-27b):** Rapid 27-billion parameter multi-lingual reasoning engine that triggers automatically if the primary returns an error.
3. **Tertiary Emergency (openai/gpt-oss-20b):** Ultralight 20-billion model guaranteeing sub-500ms triage synthesis under severe network degradation.

Combined with an **in-memory SHA256 prompt cache with 120-second TTL**, duplicate alerts consume zero redundant tokens and return instant triage briefings.

Real-World Showdown: Memory OFF vs. Memory ON

To demonstrate the impact of persistent memory, we built a live side-by-side comparison modal directly into the Echo-Core Command Center:





By toggling between Memory ON and Memory OFF, engineers see the difference between raw textbook hallucination and battle-tested team memory.

Live Command Center: From Triage to Post-Mortem

The Echo-Core frontend (sentinelops.html) provides a unified operational command center:

- **Interactive SVG Topology Graph:** Visualizes live service health, cascading failure paths, and inter-service dependencies.
- **Server-Sent Events (SSE):** Real-time streaming metrics (RPS, latency p95, error percentage) without polling.
- **Automated Post-Mortem Generator:** Generates an executive Markdown post-mortem with one click (/api/incidents/{id}/postmortem), detailing root causes, actions taken, and memory references.
- **Baseline SLA Health Check:** A Verify Health indicator verifying that service latency and error rates have returned to nominal operating levels before resolving.

Results and Verification

Echo-Core was verified against a comprehensive 15-test automated verification suite (tests/run_tests.py), passing with 100% clean execution:

- ✅ **Memory Layer:** Verified Hindsight Bank acme-sre hybrid retrieval and reflection.
- ✅ **Safety Guardrails:** 100% detection rate of dangerous commands across all microservices.
- ✅ **Tri-Model Failover:** Validated fallback execution under simulated API failure.
- ✅ **API & SSE:** Verified live streaming feeds, deduplication, and cascade detection.

Conclusion: The Future of SRE is Stateful

Incident response isn't about having more runbooks; it's about **remembering what happened the last time**.

By marrying **Vectorize Hindsight's biomimetic memory** with **Groq's ultra-low-latency model execution**, Echo-Core ensures that no engineering team ever pays for the same mistake twice.

- 🔗 **GitHub Repository:** https://github.com/shresthasrii/Echo-Core.git
- 🚀 **Hackathon:** Built with ❤️ for HackwithBengaluru 2.0.

Accompanying Deliverables

🎥 Deliverable 2: LinkedIn Promotional Post

markdown

🚨 It’s 3:14 AM. Production is down. Checkouts are throwing 500s.

Your natural instinct? "Restart the pods."

You run kubectl rollout restart.

45 seconds later, PostgreSQL completely locks up. A localized SEV-2 just cascaded into an existential company-wide SEV-1.

The worst part? Your team already suffered this exact same failure 3 months ago. The fix was documented in an old Slack thread nobody remembered.

We call this Institutional Amnesia. And for HackwithBengaluru 2.0, we built the cure:

Introducing **Echo-Core**: An Autonomous Incident Response Agent with Persistent Biomimetic Memory! 🧠⚡

Here’s how it transforms on-call operations:

1️⃣ Persistent Biomimetic Memory: Powered by Vectorize Hindsight (recall, reflect, retain), Echo-Core remembers every historical root cause, working fix, and critically: fixes that FAILED.

2️⃣ Tri-Model Resilience Engine: Zero-downtime inference with Groq LPUs chaining openai/gpt-oss-120b ➡️ qwen/qwen3.8-27b ➡️ openai/gpt-oss-20b.

3️⃣ 41-Pattern Safety Guardrails: Automatically intercepts dangerous commands (like pod restarts during pool saturation) before catastrophic outages happen.

4️⃣ Live SRE Command Center: Interactive dependency topology graph, real-time SSE metrics, automated Post-Mortem generator, and SLA baseline verification.

Check out our technical deep dive and open-source repository:

📖 Full Article: [Link to Medium/Dev.to]

💻 Open Source Code: https://github.com/shresthasrii/Echo-Core.git

Huge shoutout to the HackwithBengaluru 2.0 organizers and the Vectorize & Groq teams!

#DevOps #SRE #AI #Kubernetes #IncidentResponse #Groq #Vectorize #MachineLearning #HackwithBengaluru #OpenSource

---

🎬 **Deliverable 3: 3-Minute Demo Video Script**

Timestamp	Screen / Visual	Voiceover Script
0:00 – 0:30	Title Slide & Architecture Diagram	*"Hi everyone! We are presenting Echo-Core, an autonomous on-call incident response agent with persistent biomimetic memory, built for HackwithBengaluru 2.0. In modern SRE, teams suffer from institutional amnesia: the same failures recur, and engineers repeatedly attempt the same failed fixes, causing catastrophic outages. Echo-Core solves this with Vectorize Hindsight and Groq."*

Timestamp	Screen / Visual	Voiceover Script
0:30 – 1:15	Command Center UI (http://localhost:8000)	*"Here is our live Command Center. We have a real-time SVG topology graph showing service dependencies, live Prometheus metrics streamed via SSE, and cluster logs. Let's trigger Demo Scenario 20: a connection pool saturation on payments."*
1:15 – 1:55	Incident Card & Memory Toggle	*"Notice how Echo-Core immediately recalls past incident INC-1001 from team bank acme-sre. Let's click 'Compare vs Generic'. Generic AI suggests a pod restart. But Echo-Core's memory engine flags a critical warning: 'DO NOT restart pods—it causes a Postgres pool collapse!' Watch what happens if an engineer tries to run restart anyway: our pre-flight safety engine blocks it and demands confirmation!"*
1:55 – 2:35	Mitigation & Post-Mortem	*"Instead, we apply the memory-backed fix: scaling capacity and patching pool limits. In seconds, latency drops back to normal. We click 'Verify Health', confirming baseline SLA is restored, and with one click, generate an executive SRE post-mortem in Markdown, complete with timeline and historical citations."*
2:35 – 3:00	Codebase & Wrap-up	*"Echo-Core features a tri-model Groq fallback chain and passed all 15 automated verification tests. Check out our GitHub at github.com/shresthasrii/Echo-Core. Thank you!"*